

The Complete GitHub Actions & GitHub Runners Guide

From Zero to Expert — Step by Step

How to use this guide

Each step builds on the previous one. Don't skip steps even if they look simple — the vocabulary and mental model from Step 1 is used everywhere else. Every YAML example is heavily commented so you understand *why* each line exists, not just *what* it does.

PART 1: THEORY FOUNDATIONS

Step 1: What problem does GitHub Actions solve?

Before touching any YAML, you need the mental model.

The problem: When you write code, you constantly do repetitive tasks:

- Run tests every time someone pushes code
- Check code formatting/linting
- Build the project
- Deploy to a server
- Publish a package
- Send notifications

Doing this manually is slow and error-prone. You forget to run tests, you deploy the wrong branch, someone merges broken code.

The solution — Automation (CI/CD):

- **CI (Continuous Integration):** Automatically test and validate code every time it changes.
- **CD (Continuous Delivery/Deployment):** Automatically ship that code to servers/users once it passes CI.

GitHub Actions is GitHub's built-in automation platform. It watches your repository for events (push, pull request, issue opened, schedule, etc.) and runs scripts ("workflows") in response — for free (with limits) on GitHub's own servers, or on your own machines.

Think of it like this:

└ "When X happens in my repo, run Y automatically."

That's the entire concept. Everything else is detail.

Step 2: Core vocabulary (memorize this — everything else depends on it)

Term	Meaning
Workflow	The full automation process. A YAML file describing what to do and when. Lives in <code>.github/workflows/*.yaml</code>
Event	The trigger that starts a workflow (e.g., <code>push</code> , <code>pull_request</code> , <code>schedule</code>)
Job	A group of steps that run together on the same machine. A workflow can have multiple jobs.
Step	A single task inside a job — either a shell command or a reusable "Action"
Action	A reusable, packaged unit of code (like a function) — e.g., <code>actions/checkout</code> downloads your repo code
Runner	The actual machine (virtual or physical) that executes your jobs
Artifact	A file produced by a job that you want to keep/download after the workflow finishes
Secret	An encrypted variable (like a password or API key) used safely inside workflows

Visual hierarchy:

```
Workflow (the whole .yaml file)
├─ Job 1 (runs on a runner)
│   ├── Step 1 (e.g., checkout code)
│   ├── Step 2 (e.g., install dependencies)
│   └─ Step 3 (e.g., run tests)
├─ Job 2 (can run in parallel or depend on Job 1)
│   └─ Step 1 ...
```

By default, **jobs run in parallel** (independently), and **steps within a job run sequentially** (in order, on the same machine).

Step 3: What is a "Runner" exactly?

A **runner** is simply a computer (virtual machine or container) that has the GitHub Actions runner application installed. It:

1. Listens for jobs assigned to it
2. Downloads your repo code
3. Executes the steps in your job, one by one
4. Reports results back to GitHub

There are two types:

A) GitHub-hosted runners (the easy, default option)

- GitHub provides fresh virtual machines on demand (Ubuntu, Windows, macOS)
- Machine is destroyed after your job finishes — always clean
- Free tier: 2,000 minutes/month for private repos (unlimited for public repos), then billed per minute
- You just say `runs-on: ubuntu-latest` and GitHub handles everything

B) Self-hosted runners (advanced, more control)

- You install the runner software on your own machine (laptop, server, cloud VM, Docker container)
- Useful when you need: special hardware (GPUs), internal network access, custom software pre-installed, or to avoid GitHub's minute costs
- You are responsible for maintenance, security, and updates
- We'll set one up hands-on in Step 10

For now, just know: **"runs-on" in your YAML tells GitHub which type of runner to use.**

PART 2: YOUR FIRST WORKFLOW (Hands-on)

Step 4: Anatomy of a workflow file

Every workflow lives at: `.github/workflows/<any-name>.yaml`

GitHub scans this folder automatically — no registration needed. Just commit a valid YAML file there and it activates.

Here is the **minimum possible workflow**, fully commented:

yaml

```
# The display name shown in the GitHub Actions tab
name: My First Workflow

# WHEN should this workflow run? (the "event")
on: push    # Run every time someone pushes code to any branch

# WHAT should it do? (one or more jobs)
jobs:
  say-hello:      # Arbitrary ID for this job (you choose the name)
    runs-on: ubuntu-latest    # Use a GitHub-hosted Ubuntu virtual machine

    steps:        # List of steps that run in order
      - name: Print a message # Human-readable label (shows in logs)
        run: echo "Hello, GitHub Actions!" # The actual shell command to execute
```

How to create this practically:

bash

```
# 1. In your local repo, create the required folder structure
mkdir -p .github/workflows

# 2. Create the workflow file inside it
touch .github/workflows/hello.yml

# 3. Paste the YAML above into hello.yml, then commit and push
git add .github/workflows/hello.yml
git commit -m "Add first GitHub Actions workflow"
git push
```

What happens next: Go to your repository on GitHub → click the "Actions" tab → you'll see your workflow run automatically, with live logs.

Step 5: Understanding `on:` — Triggers (Events)

This is how you control *when* a workflow fires. Here are the most common ones, all commented:

yaml

```
# Trigger on multiple event types
on:
  push:                                # When code is pushed
    branches: [ main, dev ]          # ONLY trigger for pushes to these branches
    paths:                             # ONLY trigger if these files changed (optimization)
      - "src/**"
      - "!docs/**"                   # Exclamation mark = exclude this path

  pull_request:                       # When a PR is opened/updated
    branches: [ main ]               # Only PRs targeting main

  schedule:                           # Run on a timer (cron syntax, in UTC)
    - cron: "0 0 * * *"              # Every day at midnight UTC

  workflow_dispatch:                  # Adds a manual "Run workflow" button in the UI
    inputs:                           # Optional: let the user provide inputs when triggered
      environment:
        description: "Which environment to deploy to"
        required: true
        default: "staging"

  release:                            # When a GitHub release is published
    types: [ published ]
```

Practical tip: `workflow_dispatch` is extremely useful while learning — it lets you manually click "Run workflow" in the GitHub UI instead of pushing commits every time to test.

Step 6: Jobs, Steps, and the `actions/checkout` action

Your runner starts as a **completely empty machine** — it does NOT have your code by default. This surprises every beginner.

yaml

name: Build and Test

on: push

jobs:

build:

runs-on: ubuntu-latest

steps:

STEP 1: Download your repository's code onto the runner.

This is almost ALWAYS your first step in every job.

- **name:** Checkout repository

uses: actions/checkout@v4 # "uses" = run a pre-built reusable Action

@v4 pins the version – always pin versions for stability/security

STEP 2: Set up a specific language runtime

- **name:** Set up Node.js

uses: actions/setup-node@v4

with: # "with" passes inputs/parameters to the act

node-version: "20" # Install Node.js version 20

STEP 3: Run a plain shell command (no external action needed)

- **name:** Install dependencies

run: npm install # "run" = execute raw shell command(s)

STEP 4: Run your test suite

- **name:** Run tests

run: npm test

Key distinction:

- **uses:** → run someone else's pre-packaged Action (from the Marketplace or your own repo)
- **run:** → execute raw shell commands yourself

Multi-line commands:

yaml

```
- name: Multiple commands in one step
  run: |
    echo "Step A"
    echo "Step B"
    npm run build
# The pipe "|" preserves line breaks – each line runs as a separate command
```

Step 7: Environment variables and Secrets

Environment variables (non-sensitive config)

yaml

```
env:                                # Workflow-level: available to ALL jobs
  NODE_ENV: production

jobs:
  build:
    runs-on: ubuntu-latest
    env:                              # Job-level: available to all steps in this job
      BUILD_DIR: dist
    steps:
      - name: Use variables
        env:                          # Step-level: available only in this step
          GREETING: "Hello"
        run: echo "$GREETING from $NODE_ENV, output goes to $BUILD_DIR"
```

Secrets (sensitive data — passwords, tokens, API keys)

NEVER hardcode secrets in your YAML file. Anyone who reads your repo would see them.

How to add a secret (practical steps):

1. Go to your repo on GitHub
2. Settings → Secrets and variables → Actions
3. Click "New repository secret"
4. Name it (e.g., `API_KEY`) and paste the value
5. Save

How to use it in a workflow:

yaml

```
- name: Deploy using a secret
  env:
    MY_API_KEY: ${ secrets.API_KEY } # Injected securely, never print
  run: |
    curl -H "Authorization: Bearer $MY_API_KEY" https://api.example.com/d
```

GitHub automatically masks secret values in logs (shows `***`) even if you accidentally try to print them.

Step 8: Conditionals, expressions, and the `${{ }}` syntax

`${{ }}` is GitHub Actions' **expression syntax** — used to evaluate variables, functions, and conditions.

yaml

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Only run on main branch
        if: github.ref == 'refs/heads/main' # Conditional execution
        run: echo "This only runs on main"

      - name: Only run if previous step failed
        if: failure() # Built-in function
        run: echo "Something broke – sending alert"

      - name: Always run (e.g., cleanup)
        if: always() # Runs even if earlier steps fail
        run: echo "Cleanup tasks here"

      - name: Access context info
        run: |
          echo "Branch: ${ github.ref_name }"
          echo "Actor who triggered this: ${ github.actor }"
          echo "Commit SHA: ${ github.sha }"
          echo "Event name: ${ github.event_name }"
```

Common built-in contexts you'll use constantly:

- `github.*` → info about the repo/commit/event
 - `secrets.*` → your stored secrets
 - `env.*` → environment variables
 - `matrix.*` → matrix build values (Step 9)
 - `steps.<step_id>.outputs.*` → data passed between steps
-

PART 3: INTERMEDIATE TECHNIQUES

Step 9: Matrix builds (run the same job with multiple configurations)

Instead of copy-pasting a job for every Node version or OS, use a **matrix**:

```
yaml
jobs:
  test:
    runs-on: ${{ matrix.os }}      # OS comes from the matrix itself

    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node-version: [18, 20, 22]
        fail-fast: false          # If one combination fails, don't cancel the rest
        max-parallel: 3           # Limit how many run simultaneously

    steps:
      - uses: actions/checkout@v4

      - name: Set up Node ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}

      - run: npm test
```

This single job definition automatically expands into **9 parallel jobs** (3 OS × 3 Node versions). Extremely powerful for cross-platform/cross-version testing.

Step 10: Setting up a Self-Hosted Runner (hands-on)

Now let's actually configure your own machine as a runner.

When to use this: You need custom hardware, internal/private network access, pre-installed software, or you're exceeding free GitHub-hosted minutes.

Practical setup (Linux example):

```
bash

# STEP 1: Go to your repo on GitHub
# Settings → Actions → Runners → "New self-hosted runner"
# GitHub will show YOU a specific download link and TOKEN – copy them.

# STEP 2: Create a folder for the runner on your machine
mkdir actions-runner && cd actions-runner

# STEP 3: Download the runner package (URL comes from GitHub's UI, version change)
curl -o actions-runner-linux-x64.tar.gz -L \
  https://github.com/actions/runner/releases/download/vX.X.X/actions-runner-linux-x64-vX.X.X.tar.gz

# STEP 4: Extract it
tar xzf ./actions-runner-linux-x64.tar.gz

# STEP 5: Configure the runner – connects it to YOUR repo using the token from
./config.sh --url https://github.com/YOUR-USERNAME/YOUR-REPO --token YOUR_TOKEN
# It will ask you to name the runner and assign "labels" (tags you can target in workflows)

# STEP 6: Start the runner (this makes it listen for jobs)
./run.sh
# Keep this terminal open – the runner is now "online" and waiting for work

# OPTIONAL: Install it as a background service so it survives reboots
sudo ./svc.sh install
sudo ./svc.sh start
```

Using your self-hosted runner in a workflow:

```
yaml

jobs:
  build:
    runs-on: self-hosted      # Instead of "ubuntu-latest", target YOUR machine
    # You can also target by label: runs-on: [self-hosted, linux, gpu]
    steps:
      - uses: actions/checkout@v4
      - run: echo "This runs on MY OWN machine, not GitHub's cloud"
```

⚠️ **Security warning (important):** Never use self-hosted runners on **public repositories**. Anyone can open a pull request containing malicious code, and if your workflow auto-runs on PRs, that code executes directly on your machine. Self-hosted runners are safe primarily for **private repos** you fully control.

Step 11: Caching dependencies (speed optimization)

Reinstalling dependencies (like `node_modules`) every run wastes time. Cache them:

```
yaml
- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: ~/.npm                # Folder to cache
    key: npm-${{ hashFiles('package-lock.json') }}
    # The "key" changes automatically whenever package-lock.json changes,
    # so cache is invalidated correctly when dependencies actually change
    restore-keys: |
      npm-                      # Fallback: use a partial match
- run: npm install             # Now much faster if cache was restored
```

Step 12: Artifacts — saving files produced by your workflow

Artifacts let you keep build outputs (test reports, compiled binaries, screenshots) after the job ends.

yaml

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm run build    # Produces a "dist/" folder, for example

      - name: Upload build output
        uses: actions/upload-artifact@v4
        with:
          name: production-build    # Name shown in the Actions UI download link
          path: dist/               # Folder/file to upload
          retention-days: 7         # Auto-delete after 7 days (saves storage)

# A SEPARATE job can download it later (e.g., for deployment)
deploy:
  needs: build                    # Wait for "build" job to finish first
  runs-on: ubuntu-latest
  steps:
    - name: Download build output
      uses: actions/download-artifact@v4
      with:
        name: production-build
    - run: echo "Now I have the files, ready to deploy"
```

Notice `needs: build` — this creates a **dependency chain**, forcing `deploy` to wait until `build` succeeds. Without `needs`, jobs run in parallel by default.

Step 13: Passing data between steps and jobs

Between steps (same job) — using `outputs`

yaml

```
- name: Generate a version number
  id: versioning                    # Give this step an ID so we can reference it
  run: echo "version=1.2.3" >> "$GITHUB_OUTPUT"
  # $GITHUB_OUTPUT is a special file — writing key=value here creates an output

- name: Use that version
  run: echo "The version is ${ steps.versioning.outputs.version }"
```

Between jobs — job-level outputs

yaml

```
jobs:
  job-one:
    runs-on: ubuntu-latest
    outputs:
      my-output: ${ steps.step-id.outputs.version } # Bubble the step output
    steps:
      - id: step-id
        run: echo "version=2.0.0" >> "$GITHUB_OUTPUT"

  job-two:
    needs: job-one # Must wait for job-one
    runs-on: ubuntu-latest
    steps:
      - run: echo "Version from job-one is ${ needs.job-one.outputs.my-output }
```

PART 4: ADVANCED PATTERNS

Step 14: Reusable workflows (DRY — don't repeat yourself)

If multiple repos or workflows share the same logic, extract it into a **reusable workflow**.

File: `.github/workflows/reusable-build.yml`

```

yaml

name: Reusable Build Workflow

on:
  workflow_call:           # This special trigger makes it "callable"
    inputs:
      node-version:
        type: string
        required: false
        default: "20"
    secrets:
      NPM_TOKEN:
        required: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ inputs.node-version }
      - run: npm install
      - run: npm run build

```

Calling it from another workflow:

```

yaml

name: Main CI

on: push

jobs:
  call-build:
    uses: ../github/workflows/reusable-build.yml # Path to the reusable workflow
    with:
      node-version: "22"
    secrets:
      NPM_TOKEN: ${ secrets.NPM_TOKEN }

```

Step 15: Composite Actions (reusable step sequences)

While reusable workflows share entire jobs, **composite actions** let you package a *sequence of steps* into a single custom action.

File: `.github/actions/setup-project/action.yml`

```
yaml

name: "Setup Project"
description: "Checks out code and installs dependencies"

inputs:
  node-version:
    description: "Node version to use"
    required: false
    default: "20"

runs:
  using: "composite"          # Marks this as a composite action
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: ${ inputs.node-version }
    - run: npm install
      shell: bash              # REQUIRED for every "run" step inside composite
```

Using it:

```
yaml

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: ../.github/actions/setup-project  # Local path to your composite
        with:
          node-version: "22"
      - run: npm test
```

Step 16: Permissions and Security Hardening

By default, workflows get a `GITHUB_TOKEN` with broad permissions. Restrict it to the minimum needed:

yaml

```
permissions:
  contents: read          # Only allow reading repo contents
  pull-requests: write    # Allow commenting on PRs, nothing else

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Comment on PR
        run: gh pr comment ${{ github.event.pull_request.number }} --body "Build
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}    # Auto-provided, no setup nee
```

Other hardening practices:

yaml

```
# Pin third-party actions to a FULL COMMIT SHA, not just a version tag,
# to prevent supply-chain attacks (a tag can be moved to malicious code l
- uses: actions/checkout@8f4b7f84864484a7bde6b3b8b0b7b4b1e1e1e1e1 # inst
```

Always review third-party Actions before using them, and prefer official/verified publishers in the Marketplace.

Step 17: Concurrency control (cancel outdated runs)

Prevent wasted resources when multiple pushes happen quickly (e.g., someone pushes 3 times in a minute):

yaml

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}    # Unique group per branch+w
  cancel-in-progress: true                          # Cancel older runs in the
```

This means: if you push again while a previous run is still going on the same branch, GitHub cancels the old run and only keeps the latest.

Step 18: A realistic, complete production-style workflow

Putting it all together — a real CI/CD pipeline:

yaml

name: CI/CD Pipeline

on:

push:

branches: [main]

pull_request:

branches: [main]

permissions:

contents: read

concurrency:

group: \${{ github.workflow }}-\${{ github.ref }}

cancel-in-progress: true

env:

NODE_VERSION: "20"

jobs:

---- JOB 1: Lint and test code quality ----

lint-and-test:

runs-on: ubuntu-latest

strategy:

matrix:

node-version: [18, 20]

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with:

node-version: \${{ matrix.node-version }}

- name: Cache dependencies

uses: actions/cache@v4

with:

path: ~/.npm

key: npm-\${{ hashFiles('package-lock.json') }}

- run: npm ci # "ci" = clean, reproducible install (better)

- run: npm run lint

- run: npm test

---- JOB 2: Build the project (only after tests pass) ----

build:

runs-on: ubuntu-latest

Wait for tests to succeed first

```

needs: lint-and-test # wait for tests to succeed first
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: ${{ env.NODE_VERSION }}
  - run: npm ci
  - run: npm run build

  - uses: actions/upload-artifact@v4
    with:
      name: build-output
      path: dist/

# ---- JOB 3: Deploy (only on main branch, only after build succeeds) ----
deploy:
  needs: build
  if: github.ref == 'refs/heads/main' # Never deploy from pull requests
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
      with:
        name: build-output

    - name: Deploy to production
      env:
        DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
      run: |
        echo "Deploying build to production server..."
        # your real deploy command would go here (rsync, scp, cloud CLI, etc.

```

This single file demonstrates: triggers, permissions, concurrency, matrix testing, caching, dependent jobs, artifacts, conditional deployment, and secrets — everything you learned above, working together.

PART 5: BEGINNER MISTAKES (read this carefully)

1. **Forgetting** `actions/checkout@v4` — Your runner starts empty. Without this step, `npm install`, `npm test`, etc. will fail because there's no code to work with.
2. **Not pinning action versions** — Using `uses: actions/checkout` without a version tag can silently break your pipeline when the action updates. Always pin (`@v4`), or better, a commit SHA for security-sensitive workflows).
3. **Hardcoding secrets directly in YAML** — Never write API keys, passwords, or tokens as plain text in your workflow file. Always use `secrets.*`. Once committed, secrets exist in git history forever, even if you delete them later.
4. **Confusing job-level parallelism** — Beginners expect jobs to run in order by default. They don't — jobs run in **parallel** unless you explicitly add `needs:`.
5. **Wrong YAML indentation** — YAML is whitespace-sensitive. A misplaced space breaks the entire workflow. Always use spaces (not tabs), and keep indentation consistent (2 spaces is the convention).
6. **Using self-hosted runners on public repos** — This is a serious security risk. Anyone can submit a PR that runs malicious code directly on your machine.
7. **Not scoping** `permissions:` — Leaving default (often broad) `GITHUB_TOKEN` permissions is an unnecessary security risk. Always set the minimum required.
8. **Forgetting** `if: always()` **for cleanup steps** — If a step fails, all subsequent steps are skipped by default. If you need something to run regardless (like uploading logs), you must explicitly add `if: always()`.
9. **Not using caching → slow pipelines** — Reinstalling all dependencies every single run wastes minutes (and money on private repos). Use `actions/cache`.
10. **Testing everything by pushing to GitHub** — This is slow and clutters your commit history. Use `workflow_dispatch` for manual test triggers, and consider tools like `act` to run workflows locally before pushing.
11. **Not checking the "Actions" tab logs carefully** — Every step's output is fully visible there. Beginners often guess at failures instead of reading the actual error message in the logs.
12. **Overusing** `schedule` **with tight cron intervals** — Cron-triggered workflows on GitHub-hosted runners can be delayed during high load; don't rely on second-level precision.
13. **Not using** `npm ci` **in pipelines** — `npm install` can update `package-lock.json` unexpectedly; `npm ci` guarantees a clean, reproducible install matching the lock file exactly — always prefer it in CI.
14. **Mixing up** `run` **vs** `uses` — `run` executes shell commands; `uses` calls a packaged action. Trying to put action-style syntax (`with:`) under a `run` step (or vice versa) causes YAML errors.

yaml

```
name: <workflow name>
on: <event(s)>                # push, pull_request, schedule, workflow_dispatch
permissions: <scopes>         # security: restrict GITHUB_TOKEN
concurrency: <group settings> # cancel outdated runs
env: <global variables>

jobs:
  <job-id>:
    needs: <other-job-id>      # dependency ordering
    if: <condition>            # conditional execution
    runs-on: <runner>          # ubuntu-latest / windows-latest / macos-latest
    strategy:
      matrix: <combinations>   # parallel variations
    outputs: <job-level outputs>
    steps:
      - name: <label>
        id: <step-id>           # for referencing outputs later
        uses: <action>@<version> # OR
        run: <shell command>
        with: <inputs for "uses">
        env: <step-level variables>
        if: <step-level condition>
```

You now have the complete path: theory → first workflow → triggers → jobs/steps → secrets → conditionals → matrix builds → self-hosted runners → caching → artifacts → passing data → reusable workflows → composite actions → security → concurrency → a full production pipeline → common mistakes to avoid.

Suggested practice path: Recreate Step 4 in your own repo, then work through Steps 6-13 one at a time in that same repo, committing after each change so you can see the Actions tab evolve. By the time you reach Step 18, you'll have built it yourself rather than just read it.